Intro

Convex optimization is present in many applications.

The gradient of the objective function is of the utmost importance. It is involved in numerical approximation methods, inverse problems, and optimal transport.

The article describes a simple but powerful method to **learn a monotone gradient function via a deep learning network**.

Network design ensures that the represented function is a monotone gradient field, without need to further optimize or constrain the training.

This "gradient by design" is enforced by ensuring that the **Jacobian of the network is positive definite**, which is the second-order convexity condition for the parent objective function.

Optimal Transport Survival Kit

General measures : set of Radon measures

We consider here the set of Radon measures $\mathcal{M}(\mathcal{X})$ over a set $\mathcal{X}$, and the set $\mathcal{C}(\mathcal{X})$ of continuous functions over $\mathcal{X}$. $\mathcal{M}(\mathcal{X})$ is the set of measures α over $\mathcal{X}$ that verify: $\forall f \in \mathcal{C}(\mathcal{X}), \int_{\mathcal{X}} f(x)d\alpha(x) \in \mathbb{R}$.

We note $\mathcal{M}_+(\mathcal{X})$ the subset of $\mathcal{M}(\mathcal{X})$ composed of positive measures, and $\mathcal{M}_+^1(\mathcal{X})$ the subset of probability measures (ie $\int_{\mathcal{X}} f(x)d\alpha(x) = 1$).

If α has a density wrt the Lebesgue measure, ie $d\alpha(x) = \rho_\alpha(x)$, then $\forall f \in\in \mathcal{C}(\mathbb{R}^d)$, $\int_{\mathbb{R}^d} f(x)d\alpha(x) = \int_{\mathbb{R}^d} f(x)\rho_\alpha(x)dx$.

G. Peyré and M. Cuturi, Computational Optimal Transport, arXiv:1803.00567, Mar. 2020. doi: 10.48550/arXiv.1803.00567. [Online]. Available: http://arxiv.org/ abs/1803.00567

Optimal Transport Survival Kit

Push Forward

Let $\mathcal{X}$ and $\mathcal{Y}$ be two measurable sets, $T : \mathcal{X} \longrightarrow \mathcal{Y}$ continuous.
For $\alpha \in \mathcal{M}(\mathcal{X})$, the **push forward measure** $\beta = T_{\#}\alpha \in \mathcal{M}(\mathcal{Y})$ satisfies :

$$\forall h \in \mathcal{C}(\mathcal{Y}), \int_{\mathcal{Y}} h(y)d\beta(y) = \int_{\mathcal{X}} h(T(y))d\alpha(x)$$

or equivalently :

$$\begin{aligned} \forall B \in \mathcal{Y} \text{ measurable}, \beta(B) &= \alpha(\{x \in \mathcal{X} \text{ s.t. } T(x) \in B\}) \\ &= \alpha(T^{-1}(B)) \end{aligned}$$

The application T is often refered to as the *transport map* from distribution α to distribution β.

G. Peyré and M. Cuturi, Computational Optimal Transport, arXiv:1803.00567, Mar. 2020. doi: 10.48550/arXiv.1803.00567. [Online]. Available: http://arxiv.org/ abs/1803.00567

Optimal Transport Survival Kit

Monge problem

The **Monge problem** consists in finding a continuous map T from $\mathcal{X}$ to $\mathcal{Y}$ that transports (ie *pushes forward*) a measure α to a measure β, minimizing a transport cost $c(x, y)$.

Formally :

$$\text{for } \alpha \in \mathcal{M}(\mathcal{X}), \beta \in \mathcal{M}(\mathcal{Y}),$$
$$\text{for } c : \mathcal{X} \times \mathcal{Y} \longrightarrow \mathbb{R}$$
$$\text{find } T^* : \mathcal{X} \longrightarrow \mathcal{Y} \text{ continuous st}$$
$$T^* = \underset{T}{\operatorname{argmin}} \{ \int_{\mathcal{X}} c(x, T(x)) d\alpha(x), \text{ st } T_{\#}\alpha = \beta \}$$

This formulation may or may not have a solution, as it requires a point-to-point transport.

G. Peyré and M. Cuturi, Computational Optimal Transport, arXiv:1803.00567, Mar. 2020. doi: 10.48550/arXiv.1803.00567. [Online]. Available: http://arxiv.org/ abs/1803.00567

Optimal Transport Survival Kit

Kantorovitch relaxation

Kantorovitch symetrizes the problem and relaxes it by allowing transport between distributions, and not only between points.

We note the set of joint probability measures over $\mathcal{X} \times \mathcal{Y}$ with marginales α over $\mathcal{X}$ and β over $\mathcal{Y}$ by:

$$\mathcal{U}(\alpha, \beta) = \{\pi \in \mathcal{M}_+^1(\mathcal{X} \times \mathcal{Y})\} \, \text{st} P_{\mathcal{X}\#\pi} = \alpha, P_{\mathcal{Y}\#\pi} = \beta\}$$

The Kantorovitch formulation is :

$$\mathcal{L}_c(\alpha, \beta) = \min_{\pi \in \mathcal{U}(\alpha,\beta)} \int_{\mathcal{X}\times\mathcal{Y}} c(x, y) d\pi(x, y)$$

The problem is then to find a joint probability distribution, subject to two marginals, minimizing a cost function.

G. Peyré and M. Cuturi, Computational Optimal Transport, arXiv:1803.00567, Mar. 2020. doi: 10.48550/arXiv.1803.00567. [Online]. Available: http://arxiv.org/ abs/1803.00567

Optimal Transport Survival Kit

Brenier Theorem

The Brenier's theorem is an important result for $\mathcal{X} = \mathcal{Y} = \mathbb{R}^d$, when c is the squared Euclidean distance $c(x, y) = ||x - y||^2$.

The Kantorovitch formulation is then:

$$\min_{\pi \in \mathcal{U}(\alpha,\beta)} \int_{\mathbb{R}^d \times \mathbb{R}^d} ||x - y||^2 d\pi(x, y)$$

The theorem states :

- **if** at least one of the measures (say here α) is absolutely continuous wrt Lebesgue measure (ie has a density ρ_α : $d\alpha(x) = \rho_\alpha(x)$),
- **then**
  - the optimal π **exists**, **is unique**, has for support $(x, T(x))$ for $x \in \text{supp}(\alpha)$,
  - $T : \mathbb{R}^d \longrightarrow \mathbb{R}^d$ is the "Monge map" $(\pi = (\text{id}, T)_\# \alpha)$:

$$\forall h \in \mathcal{C}(\mathcal{X} \times \mathcal{Y}), \int_{\mathcal{X} \times \mathcal{Y}} h(x, y) d\pi(x, y) = \int_{\mathcal{X}} h(x, T(x)) d\alpha(x)$$

  - $T(x) = \nabla\phi(x)$ with ϕ **unique** (up to an additive constant), **convex**, with $(\nabla\phi)_\# \alpha = \beta$

G. Peyré and M. Cuturi, Computational Optimal Transport, arXiv:1803.00567, Mar. 2020. doi: 10.48550/arXiv.1803.00567. [Online]. Available: http://arxiv.org/ abs/1803.00567

Convexity 101

- **convexity first order condition** - Let $f : \mathbb{R}^d \longrightarrow \mathbb{R}$ differentiable. Then f is convex iff its gradient ∇f is monotone : $\forall x, y < \nabla f(x) - \nabla f(y), x - y >\geq 0$
- **convexity second order result** - Let $f : \mathbb{R}^d \longrightarrow \mathbb{R}$ twice differentiable. Then f is convex iff $\nabla^2 f(x) \succeq 0$ (where $\nabla^2 f(x)$ is the Hessian of f).

Let $f : \mathbb{R}^n \longrightarrow \mathbb{R}$ convex, twice differentiable. Let $g(x) = \nabla f(x)$. Then:

$$f \text{ convex } \iff \nabla^2 f(x) = J_g(x) \succeq 0, \ \forall x \in \mathbb{R}^n$$

where $J_g(x)$ is the Jacobian of g taken in x.

If $g(x) = g_\theta(x)$ is a neural network, then

$$g = \nabla f \iff J_{g_\theta}(x) \succeq 0, \forall x \in \mathbb{R}^n$$

The network g_θ represents a gradient field -and an optimal transport map for the L2 cost- iff its Jacobian is positive semi definite everywhere.

Convex Optimization – Boyd and Vandenberghe. [Online]. Available: https://stanford. edu/~boyd/cvxbook/

Cascade Network : C-MGN

The first model is **C-MGN** :

$$
\begin{aligned}
z_0 &= Wx + b_0 \text{ first layer} \\
z_l &= Wx + \sigma_l(z_{l-1}) + b_l \text{ next layers} \\
\text{C-MGN}(x) &= W^T \sigma_L(z_{L-1}) + V^T V x + b_L
\end{aligned}
$$

if $\forall l, \sigma_l(.)$ is a differentiable, element-wise activation function that is monotically increasing, then $\forall x, J_{C-MGN}(x) \succeq 0$

Finally $J_{C-MGN}(x) = W^T A W + V^T V$ with $A \succeq 0$.

So $J_{C-MGN}(x) \succeq 0$.

Which proves that **C-MGN is the gradient of a convex function**.

Cascade Network : C-MGN - Jacobian calculation

We calculate the Jacobian of C-MGN$(x) = W^T \sigma_L(z_{L-1})(x) + V^T V x + b_L$.
First,

$$
\begin{aligned}
J_{\sigma_L \circ z_{L-1}}(x) &= J_{\sigma_L}(z_{L-1})(x).J_{z_{L-1}}(x) \\
\text{with } z_{L-1}(x) &= Wx + \sigma_{L-1}(z_{L-2}) + b_{L-1} \\
J_{z_{L-1}}(x) &= W + J_{\sigma_{L-1} \circ z_{L-2}}(x) \\
J_{\sigma_L \circ z_{L-1}}(x) &= J_{\sigma_L}(z_{L-1}).(W + J_{\sigma_{L-1} \circ z_{L-2}}(x)) \\
&= J_{\sigma_L}(z_{L-1})W + J_{\sigma_L}(z_{L-1}).J_{\sigma_{L-1} \circ z_{L-2}}(x) \\
&= J_{\sigma_L}(z_{L-1})W + J_{\sigma_L}(z_{L-1}) \left(J_{\sigma_{L-1}}(z_{L-2})(W + J_{\sigma_{L-2} \circ z_{L-3}})(x)\right) \\
&= ... \\
&= \left(J_{\sigma_L}(z_{L-1}) + J_{\sigma_L}(z_{L-1})J_{\sigma_{L-1}}(z_{L-2}) + ... + J_{\sigma_L}(z_{L-1}) \times ... \times J_{\sigma_1}(z_0)\right) W \\
&= \left(\sum_{l=1}^{L} \prod_{i=l}^{L} J_{\sigma_i}(z_{i-1})\right) W
\end{aligned}
$$

So :

$$J_{C-MGN}(x) = W^T \left(\sum_{l=1}^{L} \prod_{i=l}^{L} J_{\sigma_i}(z_{i-1})\right) W + V^T V$$

The activation functions are $\sigma_i(.)$ are monotically increasing element wise :

$$\sigma_i(x) = \begin{pmatrix} \sigma_i(x_1) \\ ... \\ \sigma_i(x_p) \end{pmatrix}$$

$$J_{\sigma_i}(x) = \begin{pmatrix} \frac{\partial \sigma_i}{\partial x_1} & 0 & ... & 0 \\ 0 & \frac{\partial \sigma_i}{\partial x_2} & ... & 0 \\ 0 & ... & ... & 0 \\ 0 & ... & ... & \frac{\partial \sigma_i}{\partial x_p} \end{pmatrix} \succeq 0$$

So $\sum_{l=1}^{L} \prod_{i=l}^{L} J_{\sigma_i}(z_{i-1}) \succeq 0$.
Finally $J_{C-MGN}(x) = W^T A W + V^T V$ with $A \succeq 0$.
So $J_{C-MGN}(x) \succeq 0$.

Cascade Network : C-MGN - Key “tricks”

Jacobian of composed function…

$$J_{f \circ g}(x) = J_f(g(x)).J_g(x)$$

PyTorch class to share weight matrix W across layers

Parameter

CLASS torch.nn.parameter.Parameter(*data=None*, *requires_grad=True*) [SOURCE]

A kind of Tensor that is to be considered a module parameter.

Module Network : M-MGN

The model is:

$$z_k = W_k x + b_k$$

$$M - MGN(x) = a + V^T V x + \sum_{k=1}^{K} s_k(z_k) W_k^T \sigma_k(z_k)$$

where $s_k(.)$ is a *scalar* function $s_k : \mathbb{R}^n \longrightarrow \mathbb{R}$ that is **convex, twice differentiable, non-negative**, and that verifies:

$$\sigma_k(.) = \nabla s_k(.)$$

$$J_{M-MGN}(x) = V^T V + \sum_{k=1}^{K} \left(s_k(z_k) W_k^T J_{\sigma_k}(z_k) W_k + (W_k^T \sigma_k(z_k))(W_k^T \sigma_k(z_k))^T \right)$$

M-MGN is the gradient of a convex function.

Module Network : M-MGN - Jacobian calculation

Then, if we write:

$$g_k(x) = s_k(z_k) W_k^T \sigma_k(z_k)(x)$$

The Jacobian writes:

$$\begin{aligned} J_{g_k}(x) &= J_{s_k(z_k)W_k^T}.\sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k(z_k)}(x) \\ &= J_{s_k(z_k)} W_k^T \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k}(z_k) J_{z_k}(x) \end{aligned}$$

We have $z_k = W_k x + b_k$, so $J_{z_k} = W_k$. And also:

$$J_{s_k} = \begin{pmatrix} \nabla s_k|_1^T \\ \nabla s_k|_2^T \\ \dots \\ \nabla s_k|_n^T \end{pmatrix} = \begin{pmatrix} \sigma_1^T \\ \sigma_2^T \\ \dots \\ \sigma_n^T \end{pmatrix} = \sigma_k^T$$

Finally:

$$\begin{aligned} J_{g_k}(x) &= J_{s_k(z_k)} W_k^T \sigma_k(z_k) + s_k(z_k) W_k^T J_{\sigma_k}(z_k) W_k \\ J_{s_k(z_k)} &= J_{s_k}(z_k).J_{z_k)} = \sigma_k(z_k)^T W_k \\ J_{g_k}(x) &= (\sigma_k(z_k)^T W_k)(W_k^T \sigma_k(z_k)) + s_k(z_k) W_k^T J_{\sigma_k}(z_k) W_k \end{aligned}$$

Finally:

$$J_{M-MGN}(x) = V^T V + \sum_{k=1}^{K} \left(s_k(z_k) W_k^T J_{\sigma_k}(z_k) W_k + (W_k^T \sigma_k(z_k))(W_k^T \sigma_k(z_k))^T \right)$$

$s_k(.)$ is convex, so $J_{\sigma_k} \succeq 0$. $s_k(.)$ is also scalar non-negative, so the first term $s_k(z_k) W_k^T J_{\sigma_k}(z_k) W_k$ is PSD. So are $(\sigma_k(z_k)^T W_k)(W_k^T \sigma_k(z_k))$ and $V^T V$. Finally, $J_{M-MGN} \succeq 0$.

So **M-MGN is the gradient of a convex function**.

XP1 : Toy Case Gradient Field

Goal : learn basic gradient field

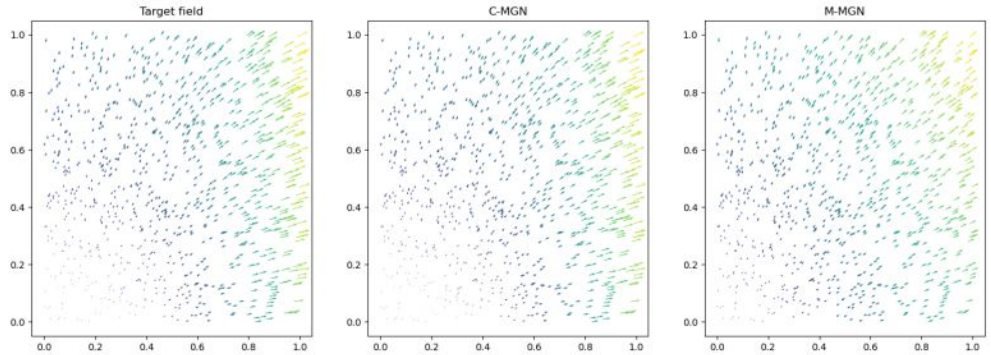
The convex function is $f : \mathbb{R}^2 \to \mathbb{R}$:

$$f(x_1, x_2) = x_1^4 + \frac{x_1^2}{2} + \frac{x_1 x_2}{2} + \frac{3x_2^2}{2} - \frac{x_2^3}{3} \tag{1}$$

The gradient field to learn, ie the transport map, is:

$$T(x_1, x_2) = \begin{pmatrix} 4x_1^3 + \frac{1}{2}x_2 + x_1 \\ 3x_2 - x_2^2 + \frac{1}{2}x_1 \end{pmatrix} \tag{2}$$

Both a C-MGN and a M-MGN networks are trained, with a L1 loss between the model computations and the target fields. Both architectures train well.



XP2 : OT between Gaussians

Goal : calculate OT coupling between two Gaussians. Use loss functions with close-form solutions

Here, the two architectures learn a coupling between a source Gaussian ($\mathcal{N}(0, 2 * \mathrm{Id}_2)$) and a target Gaussian.

Two different losses are used, that have closed form solutions for two Gaussians : the Wasserstein-2 distance, and the KL-divergence. Recall that the W2 distance and KL-divergence between two Gaussians $\mathcal{N}_1(\mu_1, \Sigma_1)$ and $\mathcal{N}_2(\mu_2, \Sigma_2)$ are:

$$\mathcal{W}(\mathcal{N}_1, \mathcal{N}_2) = ||\mu_1 - \mu_2||_2^2 + \mathrm{Trace}\left(\Sigma_1 + \Sigma_2 - 2 * (\Sigma_2^{\frac{1}{2}} \Sigma_1 \Sigma_2^{\frac{1}{2}})^{\frac{1}{2}}\right)$$

$$\mathbb{KL}(\mathcal{N}_1 || \mathcal{N}_2) = \frac{1}{2}\left(\mathrm{Trace}(\Sigma_2^{-1}\Sigma_1) - k + (\mu_2 - \mu_1)^T \Sigma_2^{-1}(\mu_2 - \mu_1) + \ln(\frac{|\Sigma_2|}{|\Sigma_1|})\right)$$

where k is the dimensionnality of the two Gaussians.

Overall, the C-MGN trains faster and easier than the M-MGN. The training of the M-MGN with the KL-divergence is difficult.

XP3 : Domain Adaptation - “from dusk till dawn”

Goal : Transport color distribution of a “daytime” image to “sunset” or “night” color distribution

Here, the color distribution of a daytime image is mapped to the color distribution of a target image (either "sunset" or "night"), both distributions approximated by a 3D Gaussian. A C-MGN with a Wasserstein-2 loss is used.

Figure 2: Day to Sunset

Figure 3: Day to Night

THANK YOU

APPENDIX

Proofs - Convexity

A.1 Result 1 - Convexity

Let $f : \mathbb{R}^d \longrightarrow \mathbb{R}$ differentiable. Then f is convex iff its gradient ∇f is monotone : $\forall x, y < \nabla f(x) - \nabla f(y), x - y >\geq 0$.

Proof

f is convex iff $\forall x, y \in \mathbb{R}^d, f(y) \geq f(x) + < \nabla f(x), y - x >$. Substituting x and y, we also have $f(x) \geq f(y) + < \nabla f(x), x - y >$. Adding the two equations, we get $0 \geq < y - x, \nabla f(x) - \nabla f(y) >$, or $< \nabla f(y) - \nabla f(x), y - x > \geq 0$

A.2 Result 2 - Convexity

Let $f : \mathbb{R}^d \longrightarrow \mathbb{R}$ twice differentiable. Then f is convex iff $\nabla^2 f(x) \succeq 0$ (where $\nabla^2 f(x)$ is the Hessian of f.

Proof

First, assume $n = 1$. $f : \mathbb{R} \longrightarrow \mathbb{R}$ twice differentiable.

- if f is convex, then the first order condition writes

$$\begin{aligned} \forall x, y \in \mathrm{dom} f, y > x \implies f'(x)(y - x) \leq f(y) - f(x) \leq f'(y)(y - x) \\ \implies \frac{f'(y) - f'(x)}{y - x} \geq 0 \\ \implies f''(x) = \lim_{y \longrightarrow x} \frac{f'(y) - f'(x)}{y - x} \geq 0 \end{aligned}$$

- conversely, if $\forall z \in \mathrm{dom} f, f''(z) \geq 0$. Then :

$$\begin{aligned} \forall x, y \in \mathrm{dom} f, x < y \implies 0 \leq \int_x^y f''(z)(y - z)dz \\ = f'(z)(y - z)|_x^y + \int_x^y f'(z)dz \\ = -f'(x)(y - x) + f(y) - f(x) \end{aligned}$$

which proves f is convex.

Second, for $n \geq 2$, f is convex iff it is convex on all lines, ie $g(t) = f(x_0 + tv)$ convex in t $\forall x_0 \in \mathrm{dom} f, \forall v$. So f is convex iff

$$\begin{aligned} g''(t) = v^T \nabla^2 f(x_0 + tv) v \geq 0, \ \forall x_0 \in \mathrm{dom} f, \forall v \in \mathbb{R}^n, \forall t \, \mathrm{st} \, x_0 + tv \in \mathrm{dom} f \\ \iff \nabla^2 f(x) \succeq 0 \end{aligned}$$